

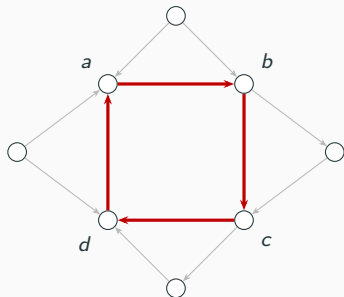
Štetje 4-ciklov v polno dinamičnih grafih

Z vidika cikličnih stikov

Govorec 1 Govorec 2 Govorec 3 Govorec 4

Seminar iz algoritmov – poskusna predstavitev za PODS 2025

Motivacijski primer: ciklični vzorci v grafu transakcij



računi (vozlišča), transakcije (puščice)

Z računa a gre nakazilo na račun b , z b na c , s c na d in z d nazaj na a – to je **4-cikel v grafu transakcij** in značilen vzorec pri odkrivanju goljufij.

Formalno gre za **ciklično konjunktivno poizvedbo**: štiri relacije, sklenjene v zanko. Število rezultatov poizvedbe je natanko število 4-ciklov v grafu.

Enaka struktura se pojavlja v številnih primerih – cikli citatov, skupine, ki si vzajemno sledijo na družbenih omrežjih, priporočilne zanke.

Pri tem se graf neprestano spreminja.

Polno dinamično okolje

Vsaka nova transakcija doda eno povezavo; prav tako so dovoljena brisanja. Po *vsaki* posodobitvi mora sistem sporočiti natančno število 4-ciklov.

Ponovno izračunavanje od začetka je za velike grafe predrago. Potrebujemo hitro posodabljanje pri vsaki spremembi.

V najslabšem primeru, ne amortizirano

Strošek, ki ga merimo, je čas *najpočasnejše* posodobitve – ne pa povprečje čez celotno zaporedje posodobitev.

Sistemi podatkovnih baz so občutljivi na repne zakasnitve: že ena sama posodobitev, dolga sekundo, je za produkcijo težava, čeprav je povprečje hitro.

Cilj je torej ambiciozen: biti hitrejši od ponovnega izračuna od začetka, in to z zagotovitvijo o najslabšem primeru za vsako posamezno posodobitev.

Pokrajina: trije majhni vzorci

Doslej najboljši strošek vzdrževanja na posodobitev za posamezen vzorec:

Vzorec	Oblika	Najboljši strošek na posodobitev
Trikotnik (3-cikel)		$\sqrt{m}$
4-klika		m
4-cikel		$m^{2/3}$

- Trikotniki in 4-klike imajo jasne, naravne eksponente.
- 4-cikli imajo nenavaden eksponent $2/3$ – niti kvadratni koren niti linearno.
- Meja $2/3$ je veljala za najboljši znani rezultat več kot desetletje.

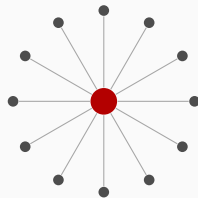
Od kod izvira meja $m^{2/3}$?

Vsak algoritem v tem prostoru izkorišča isto opazko: stopnje vozlišč so zelo različne.

- Malo **vozlišč visoke stopnje** (npr. banke, veliki trgovci); obdelava vsakega je draga.
- Veliko **vozlišč nizke stopnje**; vsako zase je poceni.

Če 4-cikle štejemo ločeno za oba razreda in prag izberemo tako, da se stroška uravnotežita, dobimo natanko $m^{2/3}$.

To je **naravna zgornja meja** za vsak kombinatorični algoritem, ki povezave obdeluje posamično.



visoka stopnja: eno vozlišče, veliko povezav



nizka stopnja: veliko vozlišč, malo povezav

Rezultat Assadija in Shaha (2025)

Assadi in Shah (PODS 2025) sta končno presegla to mejo: dosegla sta čas posodobitve $m^{2/3-\varepsilon}$ v najslabšem primeru, za neko konstanto $\varepsilon > 0$.

Nepričakovana sestavina: **hitro množenje matrik**.

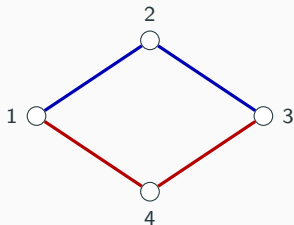
Zakaj je to presenetljivo?

- Hitro množenje matrik je **množična** operacija – učinkovita je le, če imamo na voljo dve veliki matriki in ju pomnožimo naenkrat.
- Naš problem je nasproten, **tokovni** – povezave prihajajo posamično, ena za drugo.
- V stroki je veljalo, da sta pristopa nezdružljiva.

Uskladitev: faze

Tok posodobitev razdelimo v faze. Med vsako fazo algoritem na poizvedbe odgovarja s pomočjo produkta matrik, izračunanega v prejšnji fazi, medtem ko se naslednji produkt sproti računa v ozadju – strošek je porazdeljen čez celotno fazo. Ob koncu faze se produkta zamenjata.

Zakaj množenje matrik?



Dve 2-koračni poti od 1 do 3
($1 \rightarrow 2 \rightarrow 3$ in $1 \rightarrow 4 \rightarrow 3$)
skupaj tvorita en 4-cikel.

4-cikel sestavljata dve 2-koračni poti, ki si delita isti par krajišč.

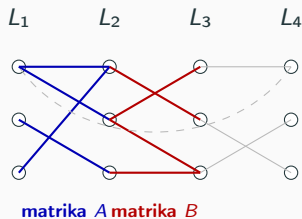
Štetje 4-ciklov se torej prevede na naslednje vprašanje:
za vsak par vozlišč (u, v) , koliko 2-koračnih poti poteka med njima?

Odgovor je tabela, indeksirana s pari vozlišč – po en vnos za vsak par.

Računanje te tabele je natanko množenje matrik.

Novi algoritem za ta korak uporablja *hitro* množenje matrik – v tem je vir asimptotične pospešitve.

Pogled cikličnega stika: od kod izvira pospešitev



$A \cdot B$ je tabela 2-koračnih poti s prejšnje prosojnice.

Vsako vozlišče 4-cikla s prejšnje prosojnice ima eno od štirih **vlog**: položaj 1, 2, 3 ali 4.

Plast L_i je množica vseh vozlišč, gledanih kot kandidati za vlogo i – konceptualno gre za štiri kopije iste množice vozlišč.

A: povezave med kandidati za vlogi 1 in 2.

B: povezave med kandidati za vlogi 2 in 3.

Produkt $A \cdot B$ nam za vsak par $(1, 3)$ pove, koliko je vmesnih vozlišč v vlogi 2 – natanko **zgornja polovica** 4-cikla s prejšnje prosojnice.

Hitro množenje matrik ta produkt izračuna strogo hitreje kot naivni pristop – prav to je bližnjica, ki prebije mejo $m^{2/3}$.

En matrični korak, podprt z ogradjem

Določili smo ključni matrični korak. Vendar ostaja temeljna napetost:

Množično proti tokovnemu

Hitro množenje matrik je **množično**: pričakuje dve celotni matriki, ki ju pomnožimo naenkrat.

Naš problem je **tokovni**: povezave prihajajo posamično, ena za drugo.

Vse ostale komponente algoritma obstajajo zato, da premostijo to vrzel:

- Razdelitev vozlišč po stopnji → nadzor nad velikostjo in obliko matrik.
- Vzdrževanje po kosih → porazdelitev množičnega stroška na veliko povezav.
- Faze → ohranjamo pripravljen produkt iz prejšnje faze, medtem ko v ozadju računamo naslednji produkt.

Vsa ta kompleksnost ima en sam namen: omogočiti, da množično množenje matrik deluje pri tokovnih posodobitvah.

Hitro množenje matrik je bistveno, ne le postransko

Bi pospešitev lahko izvirala iz katerega koli drugega dela mehanizma? Pokazali smo, da ne more.

Sistem omejitev algoritma smo analizirali pri **domnevno optimalnem** eksponentu množenja matrik.

Sklep je oster:

Če odstranimo korak hitrega množenja matrik, celotna pospešitev izgine – meja pade nazaj na prejšnji $m^{2/3}$.

Hitro množenje matrik je torej **nosilni** korak. Vse okoliško ogrodje obstaja zgolj zato, da je ta korak uporaben v tokovnem okolju.

Implementacija in eksperimentalna postavitev

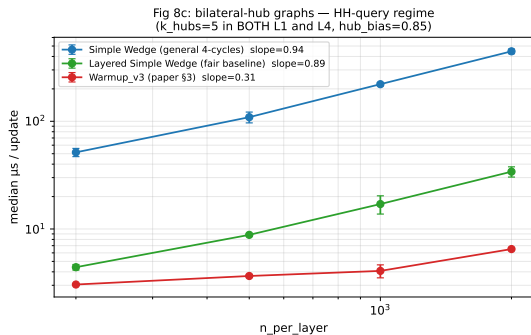
Teorija nam pove, da je matrični korak nujen. Empirično vprašanje pa je: **pri katerih vhodih se zapleten okoliški mehanizem dejansko splača?**

Trije sestavni deli, vsi v Pythonu:

- **Algoritem.** Zvesta implementacija strukture novega algoritma – štiri plasti, pragi stopenj, vzdrževanje posodobitev po kosih in osrednji korak množenja matrik.
- **Izhodišče.** Naravni, preprostejši pristop – štetje klinov na istem štiriplastnem grafu, brez mehanizma za usmerjanje po razredih.
- **Testno orodje.** Generator grafov z nadzorovano strukturo: naključni grafi, grafi z enostransko visoko stopnjo, grafi z obojestransko visoko stopnjo in grafi z enakomerno nizko stopnjo.

Oba algoritma poganjamo na enakih grafih in enakih tokovih posodobitev. Zanima nas **režim, v katerem novi algoritem zmaga**, ne pa njegov asimptotični eksponent.

Kdaj algoritem zmaga: obojestransko visoke stopnje



Novi algoritem prekaša izhodišče natanko takrat, ko ima graf **vozlišča visoke stopnje na nasprotnih straneh** cikla:



V ostalih režimih izhodišče dosega skoraj enako učinkovitost; dodatni mehanizem postane le režija.

Algoritem torej ni splošni pospeševalnik, temveč cilja na **poseben strukturni režim**, v katerem se vzpostavitev hitrega množenja matrik dejansko izplača.

Iz dveh perspektiv pridemo do istega sklepa:

- **Teoretično:** samo en korak je nosilen – hitro množenje matrik.
- **Empirično:** samo en režim resnično potrebuje celoten mehanizem – obojestransko visoke stopnje.

Oba vidika kažeta, da okoliški mehanizem obstaja zgolj zato, **da omogoči eno dobro pripravljeno množenje matrik.**

Smeri za nadaljnje delo:

- Daljši cikli – k -cikli za $k > 4$.
- Sprotno zaznavanje obojestranskega režima, sicer pa preklap nazaj na izhodiščni algoritem.
- Analiza v zaprti obliki pri *trenutnih* eksponentih množenja matrik.

Širše gledano: gre za jasen vpogled v napetost med **množičnim in tokovnim računanjem**, ki je prisotna povsod v dinamičnih podatkovnih sistemih.

Hvala.

Vprašanja?